

Неделя 1

Вопрос 1. Базовые понятия RL

Ответ: Обучение с подкреплением (RL) формализуется на основе Марковского процесса принятия решений (MDP). Базовая парадигма включает в себя следующие элементы:

- **Среда (Environment)** - система, с которой взаимодействует агент, подчиняющаяся марковскому свойству (текущее состояние полностью описывает систему для принятия решений).
- **Агент (Agent)** - сущность, принимающая решения (выбирающая действия).
- **Состояние ($s \in \mathcal{S}$)** - текущее наблюдение агента за средой.
- **Действие ($a \in \mathcal{A}$)** - выбор агента, влияющий на среду.
- **Награда ($r \in \mathbb{R}$)** - скалярный сигнал обратной связи от среды. Цель агента - максимизировать ожидаемую дисконтированную сумму наград (Return): $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$.
- **Политика ($\pi(a|s)$)** - стратегия агента, задающая распределение вероятностей выбора действия a в состоянии s .

Вопрос 2. Метод кросс-энтропии (табличный + нейронная сеть)

Ответ: Метод кросс-энтропии (CEM) - это алгоритм оптимизации, который сводит задачу RL к задаче обучения с учителем (Supervised Learning). Мы итеративно улучшаем политику, повышая вероятность действий, которые привели к высоким суммарным наградам.

Табличный случай (Tabular CEM): Применяется для небольших дискретных пространств состояний. Политика хранится в виде матрицы вероятностей. Алгоритм: 1. Сэмплируем N сессий, используя текущую политику. 2. Сортируем сессии по суммарной награде и отбираем M лучших («элиту»). 3. Обновляем вероятности политики: новая вероятность действия a в состоянии s равна доле раз, когда это действие выбиралось в этом состоянии среди всех элитных сессий.

CEM с использованием нейронной сети: Применяется при непрерывных или больших пространствах состояний, когда табличное представление невозможно. 1. Политика аппроксимируется нейронной сетью $\pi_W(a|s)$, где на вход подается состояние, а на выходе - вероятности действий. 2. Как и в табличном случае, генерируются сессии и отбирается элита. 3. Сеть обучается (градиентным спуском) максимизировать правдоподобие действий из элитных сессий:

$$W_{i+1} = W_i + \alpha \nabla \left[\sum_{s_i, a_i \in Elite} \log \pi_{W_i}(a_i | s_i) \right]$$

Таким образом, сеть решает задачу классификации (или регрессии), где признаками являются состояния из элитных сессий, а метками классов - совершенные в них действия.

Неделя 2

Вопрос 1. $V(s)$, $Q(s, a)$ - что это такое?

Ответ: Это функции ценности (Value functions), которые позволяют оценить долгосрочную перспективу нахождения в состоянии или выбора конкретного действия.

- **State-value function $V^\pi(s)$:** функция ценности состояния. Показывает ожидаемую суммарную дисконтированную награду, которую получит агент, начиная из состояния s и далее следуя политике π .

$$V^\pi(s) = \mathbb{E}_\pi[G_t | S_t = s]$$

- **Action-value function $Q^\pi(s, a)$:** функция ценности действия. Показывает ожидаемую суммарную награду, если агент в состоянии s **совершит действие a** , а уже после этого будет следовать политике π .

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t | S_t = s, A_t = a]$$

Вопрос 2. Метод Policy iteration

Ответ: Итерация по политике (Policy Iteration) - это алгоритм динамического программирования для поиска оптимальной стратегии при известной модели среды (вероятностях переходов). Метод состоит из двух чередующихся шагов:

1. **Policy Evaluation (Оценка политики):** Для текущей политики π вычисляется её функция ценности $V^\pi(s)$ для всех состояний путем итеративного решения линейных уравнений Беллмана.
2. **Policy Improvement (Улучшение политики):** Строится новая, строго детерминированная политика π' , которая на каждом шаге выбирает действие жадно (greedy) относительно вычисленных Q -значений:

$$\pi'(s) = \arg \max_a Q^\pi(s, a)$$

Согласно теореме об улучшении политики, π' будет не хуже π . Процесс повторяется до сходимости ($\pi' = \pi$).

Неделя 3

Вопрос 1. Q-learning, SARSA

Ответ: Оба алгоритма относятся к методам обучения на основе временных разностей (TD-learning) и не требуют знания модели среды.

- **Q-learning:** Является *off-policy* алгоритмом. Он обновляет оценку $Q(s, a)$ на основе максимального возможного значения в следующем состоянии, независимо от того, какое действие агент выберет в реальности:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left(r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right)$$

Это позволяет агенту выучивать оптимальную стратегию, даже если его текущее поведение носит случайный (исследовательский) характер.

- **SARSA:** Является *on-policy* алгоритмом. Он обновляет $Q(s, a)$, используя реальное следующее действие a' , которое было выбрано текущей политикой агента:

$$Q(s, a) \leftarrow Q(s, a) + \alpha (r + \gamma Q(s', a') - Q(s, a))$$

SARSA учит политику, которая безопаснее на этапе обучения, так как учитывает собственные случайные действия (например, вероятность упасть с обрыва из-за ϵ -greedy стратегии).

Вопрос 2. Exploration vs Exploitation

Ответ: Дилемма исследования и использования (Exploration vs Exploitation) является ключевой проблемой в RL.

- **Exploitation (Использование):** Агент выбирает наилучшее известное ему действие для максимизации мгновенной награды (жадная стратегия).
- **Exploration (Исследование):** Агент выбирает субоптимальные или неизвестные действия, чтобы расширить свои знания о среде и, возможно, найти более выгодную долгосрочную стратегию.

Для балансировки применяются специальные алгоритмы, такие как ϵ -greedy (с вероятностью ϵ выбирается случайное действие) или **Softmax** (действия сэмпляются пропорционально их Q-значениям с учетом параметра температуры).

Неделя 4

Вопрос 1. DQN

Ответ: Deep Q-Network (DQN) - алгоритм, объединяющий Q-learning с глубокими нейронными сетями для аппроксимации функции $Q(s, a; \theta)$ в средах с непрерывным или очень большим пространством состояний (например, пиксели экрана). Нейросеть обучается методом градиентного спуска, минимизируя среднеквадратичную ошибку (MSE) между своими текущими предсказаниями и таргетами Беллмана.

Вопрос 2. Experience replay

Ответ: Experience Replay (Буфер опыта) - техника стабилизации обучения в DQN. Переходы агента $\langle s, a, r, s' \rangle$ сохраняются в кольцевой буфер. Обучение сети производится не на последовательных кадрах (которые сильно скоррелированы), а на случайных мини-батчах, извлеченных из буфера. Это восстанавливает свойство i.i.d. (независимость и одинаковость распределения) для данных и предотвращает забывание прошлого опыта.

Вопрос 3. Target network

Ответ: Target network (Целевая сеть) решает проблему «движущейся мишени» в DQN. Если вычислять таргет $r + \gamma \max_{a'} Q(s', a'; \theta)$ используя те же веса θ , которые мы обновляем на текущем шаге градиентного спуска, обучение становится нестабильным. Для расчета таргета вводится отдельная сеть с весами θ^- , которые замораживаются и лишь периодически (раз в N шагов) копируются из основной сети.

Вопрос 4. Double DQN

Ответ: В стандартном DQN из-за использования оператора $\max$ по зашумленным оценкам нейросети возникает проблема систематической переоценки (overestimation) Q-значений. Алгоритм Double DQN устраняет этот недостаток путем разделения выбора действия и оценки его ценности. Основная сеть выбирает действие: $a^* = \arg \max_{a'} Q(s', a'; \theta)$, а целевая сеть оценивает это действие: $Q(s', a^*; \theta^-)$. Таргет принимает вид:

$$y_{ddqn} = r + \gamma Q(s', \arg \max_{a'} Q(s', a'; \theta); \theta^-)$$

Неделя 6

Вопрос 1. Policy gradient

Ответ: Policy Gradient - класс алгоритмов обучения с подкреплением, в котором напрямую оптимизируются параметры самой политики $\pi_\theta(a|s)$, а не функции ценности. Оптимизация происходит путем градиентного подъема по ожидаемой суммарной награде $J(\theta)$. Использование Policy Gradient позволяет работать с непрерывными пространствами действий и выучивать стохастические политики.

Вопрос 2. REINFORCE

Ответ: Алгоритм REINFORCE является классическим методом Policy Gradient, использующим подход Монте-Карло. Согласно Log-derivative trick, градиент целевой функции выражается как:

$$\nabla_\theta J \approx \frac{1}{N} \sum_i \sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot G_t$$

Смысл формулы: мы увеличиваем вероятность действий пропорционально тому, какую реальную суммарную награду G_t они принесли в конце эпизода. Основной недостаток - высокая дисперсия градиента, для снижения которой на практике из G_t вычитают базовую линию (baseline), например, $V(s)$.

Вопрос 3. Actor-critic

Ответ: Подход Actor-Critic комбинирует Policy-based и Value-based методы:

- **Actor (Актер)** - нейросеть, представляющая политику $\pi_\theta(a|s)$. Обновляется с помощью градиента политики.
- **Critic (Критик)** - нейросеть, аппроксимирующая функцию ценности $V_\phi(s)$ (или Q).

Критик позволяет вычислять функцию преимущества (Advantage) $A(s, a) = r + \gamma V(s') - V(s)$. Актер обучается максимизировать эту функцию. Это радикально снижает дисперсию градиентов по сравнению с алгоритмом REINFORCE и позволяет обновлять веса агента на каждом шаге (TD-подход), не дожидаясь конца эпизода.

Неделя 7

Вопрос 1. Применение RNN для генерации последовательностей

Ответ: Задачи обработки естественного языка (машинный перевод, диалоговые системы, генерация описания к изображениям) решаются с помощью архитектур Encoder-Decoder на базе RNN (или современных аналогов). При стандартном обучении с учителем (Supervised Learning) возникает проблема сдвига распределения (*Exposure Bias*). Во время тренировки модель всегда получает на вход корректные опорные токены из датасета (Teacher Forcing). Однако при инференсе (генерации) она использует собственные предыдущие предсказания. Любая ошибка генерации выводит модель в неисследованное распределение состояний, что приводит к лавинообразному накоплению ошибок.

Вопрос 2. Применение RL к дообучению моделей генерации последовательностей

Ответ: Для решения проблемы Exposure Bias и оптимизации недифференцируемых метрик (например, BLEU, ROUGE или человеческих оценок) генерация текста формулируется как процесс принятия решений (MDP). Здесь состояние - это сгенерированный до текущего момента контекст, действие - выбор следующего слова из словаря, а награда выдается в самом конце за всю сгенерированную последовательность.

- **Self-Critical Sequence Training (SCST):** Для дообучения модели применяется алгоритм REINFORCE. В качестве baseline выступает метрика текста, сгенерированного жадным образом (inference mode). Модель также сэмплирует варианты, и если сэмплированный вариант получает метрику выше жадного, его вероятность увеличивается.
- **RLHF (RL from Human Feedback):** Применяется в современных моделях (напр., InstructGPT, ChatGPT). Сначала собирается человеческая разметка для ранжирования ответов модели, на которой обучается Reward Model. Далее языковая модель (Actor) дообучается алгоритмом PPO для максимизации награды от Reward Model, с использованием штрафа за KL-дивергенцию для предотвращения чрезмерного отклонения от базового распределения слов.